TABLE OF CONTENTS

ABSTRACT ———————————————————————— 1

CHAPTERS

CHAPTER 1 – Introduction ———————————————————— 2

CHAPTER 2 – Method ———————————————————— 3

CHAPTER 3 - Test cases. ———————————————————— 5

CHAPTER 4 – Results ———————————————————-- 17

CHAPTER 5 – Summary, Conclusion ———————————————---- 19

REFERENCES ———————————————————— 21

ABSTRACT

An Operating System (OS) plays a critical role in managing computer resources and ensuring efficient execution of processes. Among its core responsibilities, CPU scheduling, memory management, and disk scheduling are fundamental components that directly influence overall system performance. This project, titled “Performance Evaluation of Scheduling, Memory, and Disk Algorithms in Operating Systems,” focuses on a comprehensive study and comparative analysis of various algorithms used in these three key areas. The primary objective of this work is to evaluate how different algorithms perform under varying workloads and system conditions. In the domain of CPU scheduling, algorithms such as First Come First Serve (FCFS), Shortest Job First (SJF), and Round Robin (RR) are analyzed based on parameters like waiting time, turnaround time, and response time. Each of these algorithms follows a distinct approach in allocating CPU time, leading to different levels of efficiency and fairness. In the context of memory management, page replacement algorithms including First In First Out (FIFO), Least Recently Used (LRU), and Optimal Page Replacement are implemented and studied. These algorithms are evaluated primarily based on the number of page faults generated and their ability to efficiently utilize available memory. The study highlights how intelligent memory management can significantly reduce overhead and improve system responsiveness. For disk scheduling, algorithms such as FCFS, Shortest Seek Time First (SSTF), and SCAN (Elevator algorithm) are examined. These algorithms determine the order in which disk I/O requests are processed, directly affecting seek time and overall disk performance. The project measures total head movement and analyzes how different strategies optimize disk access time. The implementation of these algorithms is carried out using simulation techniques, allowing controlled input scenarios and detailed performance observation

CHAPTER-1

INTRODUCTION

An An Operating System (OS) is a core component of a computer system that acts as an interface between the user and the hardware, ensuring that all resources are used efficiently and programs run smoothly. It is responsible for managing key system functions such as CPU scheduling, memory allocation, and disk operations, all of which directly impact overall system performance. CPU scheduling determines how processes share processor time, using algorithms like First Come First Serve (FCFS), Shortest Job First (SJF), and Round Robin (RR), each offering different benefits in terms of fairness, efficiency, and responsiveness.

Memory management is another essential function, as it controls how memory is allocated and freed for processes, ensuring optimal utilization and preventing conflicts. Similarly, disk scheduling plays a critical role because disk access is slower compared to memory; algorithms such as FCFS, Shortest Seek Time First (SSTF), and SCAN help reduce seek time and improve I/O performance.

This project focuses on analyzing and comparing these algorithms across CPU scheduling, memory management, and disk scheduling. By implementing and simulating different approaches, it evaluates their performance using metrics like waiting time, turnaround time, page faults, and seek time. The study highlights that no single algorithm is best in all scenarios, as performance depends on workload and system requirements. Overall, the project provides a clear understanding of the strengths and limitations of each algorithm and offers insights into designing more efficient operating systems.

CHAPTER 2

METHOD

The proposed system is designed using a **structured approach** to develop a **web-based application** capable of analyzing **operating system problems** and selecting the most suitable algorithm automatically. The methodology begins with understanding the **nature of the input** provided by the user, which may belong to **CPU scheduling**, **memory management**, or **disk scheduling**. Based on this input, the system processes the data and determines the **optimal algorithm** by considering key **performance metrics** such as **waiting time**, **page faults**, and **seek time**.

The system architecture is divided into three major components: the **User Interface (UI)**, the **Algorithm Selection Module**, and the **Execution Module**. The user interface allows users to input problem data in a simple and interactive manner. The algorithm selection module evaluates the input and compares multiple algorithms internally to determine the most **efficient solution**. This selection is based on factors such as **input size**, **workload type**, and **performance requirements**. The execution module then runs the selected algorithm and generates the output.

The implementation includes algorithms from three major domains. In **CPU scheduling**, algorithms such as **First Come First Serve (FCFS)**, **Shortest Job First (SJF)**, and **Round Robin (RR)** are implemented. In **memory management**, algorithms like **First In First Out (FIFO)**, **Least Recently Used (LRU)**, and **Optimal Page Replacement** are used. For **disk scheduling**, algorithms such as **FCFS**, **Shortest Seek Time First (SSTF)**, and **SCAN** are included. Each algorithm is implemented using **Python** and follows standard logic to compute the required performance metrics.

Once the optimal algorithm is selected, the system executes it on the given input and produces detailed results. The output includes **step-by-step execution**, **final computed values**, and **performance analysis**. In addition to the solution, the system also provides an **algorithm description** along with a **justification**, explaining why

the selected algorithm is more suitable compared to others. This improves both **problem-solving efficiency** and **conceptual understanding**.

A clean and user-friendly **web interface** is developed to enhance usability. The interface enables users to enter input data, select the problem category, and view results instantly. The system is tested with various input cases to ensure **correctness**, and the outputs are validated against **theoretical results** to maintain **accuracy** and **reliability**.

CHAPTER-3

TEST CASES/ OUTPUT

```python
from flask import Flask, render_template, request, jsonify

app = Flask(__name__)


# ────────────────────────────────────────────
#  CPU SCHEDULING ALGORITHMS
# ────────────────────────────────────────────

def fcfs_cpu(processes):
    """First-Come First-Served: sort by arrival time, process in order."""
    procs = sorted(processes, key=lambda p: p['arrival'])
    time, steps, total_wait, total_turn = 0, [], 0, 0

    for p in procs:
        if time < p['arrival']:
            time = p['arrival']          # CPU idle gap
        start = time
        finish = time + p['burst']
        wait = start - p['arrival']
        turnaround = finish - p['arrival']
        total_wait += wait
        total_turn += turnaround
        steps.append({
            'pid': p['pid'], 'arrival': p['arrival'], 'burst': p['burst'],
            'start': start, 'finish': finish,
            'waiting': wait, 'turnaround': turnaround
        })
        time = finish

    n = len(procs)
    return steps, round(total_wait / n, 2), round(total_turn / n, 2)


def sjf_cpu(processes):
    """Shortest Job First (non-preemptive): pick shortest burst among ready jobs."""
    procs = [dict(p) for p in processes]   # copy so we can mark done
    time, done, steps = 0, [], []
    total_wait, total_turn = 0, 0

    while len(done) < len(procs):
        # jobs that have arrived and aren't done
        ready = [p for p in procs if p['arrival'] <= time and p not in done]
        if not ready:
            time += 1
            continue
        p = min(ready, key=lambda x: x['burst'])   # shortest burst
        start = time
        finish = time + p['burst']
        wait = start - p['arrival']
        turnaround = finish - p['arrival']
        total_wait += wait
        total_turn += turnaround
        steps.append({
            'pid': p['pid'], 'arrival': p['arrival'], 'burst': p['burst'],
```

app.py

```python
def sjf_cpu(processes):
            'pid': p['pid'], 'arrival': p['arrival'], 'burst': p['burst'],
            'start': start, 'finish': finish,
            'waiting': wait, 'turnaround': turnaround
        })
        time = finish
        done.append(p)

    n = len(procs)
    return steps, round(total_wait / n, 2), round(total_turn / n, 2)


def round_robin_cpu(processes, quantum):
    """Round Robin: each process gets a fixed time slice (quantum)."""
    from collections import deque
    procs = sorted([dict(p) for p in processes], key=lambda x: x['arrival'])
    remaining = {p['pid']: p['burst'] for p in procs}
    arrival   = {p['pid']: p['arrival'] for p in procs}
    burst_map  = {p['pid']: p['burst'] for p in procs}

    queue = deque()
    time = 0
    in_queue = set()
    steps = []
    finish_time = {}

    # seed queue with processes that arrive at time 0
    for p in procs:
        if p['arrival'] <= time:
            queue.append(p['pid'])
            in_queue.add(p['pid'])

    while queue or any(r > 0 for r in remaining.values()):
        if not queue:
            # advance time to next arrival
            next_arrive = min(arrival[pid] for pid, r in remaining.items() if r > 0)
            time = next_arrive
            for p in procs:
                if p['arrival'] <= time and p['pid'] not in in_queue and remaining[p['pid']] > 0:
                    queue.append(p['pid'])
                    in_queue.add(p['pid'])

        pid = queue.popleft()
        run = min(quantum, remaining[pid])
        start = time
        time += run
        remaining[pid] -= run

        # enqueue newly arrived processes
        for p in procs:
            if p['arrival'] <= time and p['pid'] not in in_queue and remaining[p['pid']] > 0:
                queue.append(p['pid'])
                in_queue.add(p['pid'])

        if remaining[pid] > 0:
            queue.append(pid)   # re-queue if not done
```

app.py

```python
def round_robin_cpu(processes, quantum):
        else:
            finish_time[pid] = time

        steps.append({'pid': pid, 'start': start, 'end': time, 'ran': run})

    total_wait, total_turn = 0, 0
    summary = []
    for p in procs:
        ft = finish_time[p['pid']]
        turnaround = ft - arrival[p['pid']]
        wait = turnaround - burst_map[p['pid']]
        total_wait += wait
        total_turn += turnaround
        summary.append({'pid': p['pid'], 'arrival': arrival[p['pid']],
                        'burst': burst_map[p['pid']], 'finish': ft,
                        'waiting': wait, 'turnaround': turnaround})

    n = len(procs)
    return steps, summary, round(total_wait / n, 2), round(total_turn / n, 2)


def select_cpu_algorithm(processes, quantum):
    """
    Simple rule-based algorithm selection:
    - 1 process → FCFS (trivially optimal)
    - All same burst → FCFS (no benefit to SJF/RR)
    - Many processes (>=4) + quantum provided → Round Robin (fairness)
    - Otherwise → SJF (minimises average waiting time)
    """
    n = len(processes)
    bursts = [p['burst'] for p in processes]
    all_same = len(set(bursts)) == 1

    if n == 1 or all_same:
        return 'FCFS', 'Single process or all bursts are equal — FCFS is trivially optimal here.'
    if n >= 4 and quantum:
        return 'RR', 'Multiple processes with a time quantum provided — Round Robin ensures fair CPU sharing.'
    return 'SJF', 'Varying burst times without a quantum — SJF minimises average waiting time.'


# ──────────────────────────────────────
#  MEMORY MANAGEMENT ALGORITHMS
# ──────────────────────────────────────

def fifo_memory(pages, frames):
    """FIFO: replace the page that arrived earliest (queue order)."""
    from collections import deque
    memory, queue = [], deque()
    faults, steps = 0, []

    for page in pages:
        hit = page in memory
        evicted = None
        if not hit:
            faults += 1
```

```python
def fifo_memory(pages, frames):
            if len(memory) < frames:
                memory.append(page)
                queue.append(page)
            else:
                evicted = queue.popleft()
                memory.remove(evicted)
                memory.append(page)
                queue.append(page)
        steps.append({'page': page, 'memory': list(memory), 'fault': not hit, 'evicted': evicted})

    return steps, faults


def lru_memory(pages, frames):
    """LRU: replace the least-recently-used page."""
    memory, recent = [], []
    faults, steps = 0, []

    for page in pages:
        hit = page in memory
        evicted = None
        if hit:
            recent.remove(page)
            recent.append(page)
        else:
            faults += 1
            if len(memory) < frames:
                memory.append(page)
                recent.append(page)
            else:
                evicted = recent.pop(0)
                memory.remove(evicted)
                memory.append(page)
                recent.append(page)
        steps.append({'page': page, 'memory': list(memory), 'fault': not hit, 'evicted': evicted})

    return steps, faults


def optimal_memory(pages, frames):
    """Optimal: replace the page that will not be used for the longest time."""
    memory = []
    faults, steps = 0, []

    for i, page in enumerate(pages):
        hit = page in memory
        evicted = None
        if not hit:
            faults += 1
            if len(memory) < frames:
                memory.append(page)
            else:
                # find future uses
                future = {}
                for m in memory:
```

app.py

```python
def optimal_memory(pages, frames):
                for m in memory:
                    try:
                        future[m] = pages[i+1:].index(m)
                    except ValueError:
                        future[m] = float('inf')   # never used again
                evicted = max(future, key=future.get)
                memory.remove(evicted)
                memory.append(page)
        steps.append({'page': page, 'memory': list(memory), 'fault': not hit, 'evicted': evicted})

    return steps, faults


def select_memory_algorithm(pages, frames):
    """
    Rule-based selection:
    - frames >= unique pages → FIFO (no faults possible, FIFO is simplest)
    - small reference string (<=8) → Optimal (feasible to predict)
    - else → LRU (practical best-approximation of Optimal)
    """
    unique = len(set(pages))
    if frames >= unique:
        return 'FIFO', 'Frame count covers all unique pages — any algorithm works; FIFO is simplest.'
    if len(pages) <= 8:
        return 'Optimal', 'Small reference string — Optimal is feasible and gives the fewest faults.'
    return 'LRU', 'Large reference string — LRU is the best practical approximation of Optimal.'


# ──────────────────────────────────────
#  DISK SCHEDULING ALGORITHMS
# ──────────────────────────────────────

def fcfs_disk(requests, head):
    """FCFS: serve requests in the order they arrive."""
    steps, total = [], 0
    pos = head
    for r in requests:
        seek = abs(r - pos)
        total += seek
        steps.append({'from': pos, 'to': r, 'seek': seek})
        pos = r
    return steps, total


def sstf_disk(requests, head):
    """SSTF: always serve the closest request next."""
    remaining = list(requests)
    pos, total, steps = head, 0, []
    while remaining:
        closest = min(remaining, key=lambda r: abs(r - pos))
        seek = abs(closest - pos)
        total += seek
        steps.append({'from': pos, 'to': closest, 'seek': seek})
        pos = closest
        remaining.remove(closest)
```

```python
def scan_disk(requests, head, max_track=199):
    steps, total, pos = [], 0, head
    for r in order:
        if r == max_track and r not in requests:
            continue  # skip boundary unless it's an actual request
        seek = abs(r - pos)
        total += seek
        steps.append({'from': pos, 'to': r, 'seek': seek})
        pos = r
    return steps, total


def select_disk_algorithm(requests, head):
    """
    Rule-based selection:
    - <=3 requests → FCFS (overhead of smarter algorithms not worth it)
    - requests clustered near head (avg distance < 30) → SSTF (quick wins)
    - many spread-out requests → SCAN (minimises head movement overall)
    """
    n = len(requests)
    avg_dist = sum(abs(r - head) for r in requests) / n if n else 0

    if n <= 3:
        return 'FCFS', 'Very few requests — FCFS is simple and efficient enough.'
    if avg_dist < 30:
        return 'SSTF', 'Requests clustered near the head — SSTF minimises seek time efficiently.'
    return 'SCAN', 'Many spread-out requests — SCAN (elevator) avoids starvation and minimises total movement.'


# ──────────────────────────────────
#  EXPLANATION TEXTS
# ──────────────────────────────────

ALGO_INFO = {
    # CPU
    'FCFS': {
        'what': 'First-Come First-Served processes jobs in arrival order. Simple, no starvation, but long jobs can block short ones (convoy effect).',
        'others': 'SJF would reduce average wait, but all bursts are equal so it makes no difference. Round Robin adds unnecessary context switches here.'
    },
    'SJF': {
        'what': 'Shortest Job First picks the process with the smallest burst time. Minimises average waiting time but can starve long processes.',
        'others': 'FCFS would give higher average wait because longer jobs may run first. Round Robin needs a quantum and adds overhead without benefit here.'
    },
    'RR': {
        'what': 'Round Robin gives every process a fixed time slice (quantum). Fair and responsive, ideal when many processes share the CPU.',
        'others': 'FCFS would let long jobs monopolise the CPU. SJF without preemption could starve new short jobs that arrive later.'
    },
    # Memory
    'FIFO': {
        'what': 'First-In First-Out replaces the oldest page in memory. Simple queue, easy to implement, but can suffer Bélády\'s anomaly.',
        'others': 'LRU/Optimal are unnecessary when frames cover all unique pages — any algorithm produces zero faults.'
    },
    'LRU': {
        'what': 'Least Recently Used replaces the page unused for the longest time. Good practical approximation of Optimal.',
        'others': 'FIFO ignores recency, leading to more faults. Optimal is theoretically best but requires future knowledge — impractical in real OS.'
    },
    'Optimal': {
```

```python
@app.route('/')
def index():
    return render_template('index.html')


@app.route('/solve', methods=['POST'])
def solve():
    data = request.get_json()
    problem_type = data.get('type')

    try:
        if problem_type == 'cpu':
            result = solve_cpu(data)
        elif problem_type == 'memory':
            result = solve_memory(data)
        elif problem_type == 'disk':
            result = solve_disk(data)
        else:
            return jsonify({'error': 'Unknown problem type'}), 400

        return jsonify(result)

    except Exception as e:
        return jsonify({'error': str(e)}), 400


def solve_cpu(data):
    raw = data.get('processes', [])
    quantum = int(data.get('quantum', 0)) or None

    processes = []
    for i, p in enumerate(raw):
        processes.append({
            'pid': f"P{i+1}",
            'arrival': int(p['arrival']),
            'burst': int(p['burst'])
        })

    algo, why = select_cpu_algorithm(processes, quantum)

    if algo == 'FCFS':
        steps, avg_wait, avg_turn = fcfs_cpu(processes)
        info = ALGO_INFO['FCFS']
        return {
            'algorithm': 'FCFS (First-Come First-Served)',
            'why': why,
            'what': info['what'],
            'others': info['others'],
            'type': 'cpu_table',
            'steps': steps,
            'avg_wait': avg_wait,
            'avg_turnaround': avg_turn
        }
    elif algo == 'SJF':
        steps, avg_wait, avg_turn = sjf_cpu(processes)
        info = ALGO_INFO['SJF']
```

app.py

```python
def solve_cpu(data):
            'what': info['what'],
            'others': info['others'],
            'type': 'cpu_table',
            'steps': steps,
            'avg_wait': avg_wait,
            'avg_turnaround': avg_turn
        }
    else:  # RR
        q = quantum or 2
        timeline, summary, avg_wait, avg_turn = round_robin_cpu(processes, q)
        info = ALGO_INFO['RR']
        return {
            'algorithm': f'Round Robin (Quantum = {q})',
            'why': why,
            'what': info['what'],
            'others': info['others'],
            'type': 'cpu_rr',
            'timeline': timeline,
            'steps': summary,
            'avg_wait': avg_wait,
            'avg_turnaround': avg_turn,
            'quantum': q
        }


def solve_memory(data):
    pages  = list(map(int, str(data.get('pages', '')).split()))
    frames = int(data.get('frames', 3))

    algo, why = select_memory_algorithm(pages, frames)

    if algo == 'FIFO':
        steps, faults = fifo_memory(pages, frames)
        info = ALGO_INFO['FIFO']
    elif algo == 'Optimal':
        steps, faults = optimal_memory(pages, frames)
        info = ALGO_INFO['Optimal']
    else:  # LRU
        steps, faults = lru_memory(pages, frames)
        info = ALGO_INFO['LRU']

    return {
        'algorithm': f'{algo} Page Replacement',
        'why': why,
        'what': info['what'],
        'others': info['others'],
        'type': 'memory',
        'steps': steps,
        'faults': faults,
        'hits': len(pages) - faults,
        'total': len(pages),
        'frames': frames
    }
```

```python
def solve_memory(data):
        'steps': steps,
        'faults': faults,
        'hits': len(pages) - faults,
        'total': len(pages),
        'frames': frames
    }


def solve_disk(data):
    requests = list(map(int, str(data.get('requests', '')).split()))
    head = int(data.get('head', 0))

    algo, why = select_disk_algorithm(requests, head)

    if algo == 'FCFS':
        steps, total = fcfs_disk(requests, head)
        info = ALGO_INFO['FCFS_disk']
        algo_label = 'FCFS (First-Come First-Served)'
    elif algo == 'SSTF':
        steps, total = sstf_disk(requests, head)
        info = ALGO_INFO['SSTF']
        algo_label = 'SSTF (Shortest Seek Time First)'
    else:  # SCAN
        steps, total = scan_disk(requests, head)
        info = ALGO_INFO['SCAN']
        algo_label = 'SCAN (Elevator Algorithm)'

    return {
        'algorithm': algo_label,
        'why': why,
        'what': info['what'],
        'others': info['others'],
        'type': 'disk',
        'steps': steps,
        'total_seek': total,
        'head': head,
        'requests': requests
    }


if __name__ == '__main__':
    app.run(debug=True)
```

✦ SJF (Shortest Job First)

WHY THIS ALGORITHM?

Varying burst times without a quantum — SJF minimises average waiting time.

WHAT DOES IT DO?

Shortest Job First picks the process with the smallest burst time. Minimises average waiting time but can starve long processes.

WHY NOT THE OTHERS?

FCFS would give higher average wait because longer jobs may run first. Round Robin needs a quantum and adds overhead without benefit here.

3	6.67	3
AVG WAIT (MS)	AVG TURNAROUND (MS)	PROCESSES

GANTT CHART

P2	P3	P1
0–3	3–6	6–11

PROCESS	ARRIVAL	BURST	START	FINISH	WAITING	TURNAROUND
P2	0	3	0	3	0	3
P3	0	3	3	6	3	6
P1	0	5	6	11	6	11

Result

✦ LRU Page Replacement

WHY THIS ALGORITHM?

Large reference string — LRU is the best practical approximation of Optimal.

WHAT DOES IT DO?

Least Recently Used replaces the page unused for the longest time. Good practical approximation of Optimal.

WHY NOT THE OTHERS?

FIFO ignores recency, leading to more faults. Optimal is theoretically best but requires future knowledge — impractical in real OS.

9	4	13	30.8%
PAGE FAULTS	PAGE HITS	TOTAL REFS	HIT RATE

REF	FRAME 1	FRAME 2	FRAME 3	STATUS	EVICTED
7	7	–	–	FAULT	–
0	7	0	–	FAULT	–
1	7	0	1	FAULT	–
2	0	1	2	FAULT	7
0	0	1	2	HIT	–
3	0	2	3	FAULT	1
0	0	2	3	HIT	–
4	0	3	4	FAULT	2
2	0	4	2	FAULT	3
3	4	2	3	FAULT	0
0	2	3	0	FAULT	4
3	2	3	0	HIT	–
2	2	3	0	HIT	–

STEP 3

Result

✦ SCAN (Elevator Algorithm)

WHY THIS ALGORITHM?

Many spread-out requests — SCAN (elevator) avoids starvation and minimises total movement.

WHAT DOES IT DO?

SCAN (elevator algorithm) moves the disk arm in one direction, serving all requests along the way, then reverses.

WHY NOT THE OTHERS?

FCFS wastes movement jumping back and forth. SSTF risks starvation on a busy disk. SCAN balances fairness and efficiency.

299	53	8
TOTAL SEEK DISTANCE	INITIAL HEAD	REQUESTS

STEP	FROM	TO	SEEK DISTANCE
1	53	65	12
2	65	67	2
3	67	98	31
4	98	122	24
5	122	124	2
6	124	183	59
7	183	37	146
8	37	14	23
		Total Seek Distance	**299**

CHAPTER-4

RESULTS

The developed system was evaluated using multiple test cases across **CPU scheduling**, **memory management**, and **disk scheduling** to analyze its performance and accuracy. The results demonstrate that the system is capable of correctly identifying the **most suitable algorithm** for a given problem and producing accurate outputs.

It was observed that the system effectively selects algorithms based on **performance requirements**. In CPU scheduling scenarios, algorithms like **SJF** were chosen when minimizing **waiting time** was the priority, while **Round Robin** was preferred in cases requiring **fairness**. In memory management, **LRU** was frequently selected due to its ability to reduce **page faults**. For disk scheduling problems, **SSTF** was often identified as the optimal choice because it minimizes **seek time** by selecting the nearest request.

The performance of the algorithms was evaluated using standard metrics such as **waiting time**, **turnaround time**, **page faults**, and **total seek time**. The results obtained from the system were consistent with **theoretical expectations**. Algorithms like **SJF** and **SSTF** showed improved **efficiency** compared to **FCFS**, while **LRU** performed better than **FIFO** in most memory-related scenarios.

The system also demonstrated high **accuracy** in computation. For all tested inputs, the outputs matched **manual calculations** and expected results. The **step-by-step execution** provided by the system ensured transparency and correctness in the solution process.

Another important outcome is the system's ability to provide **explanations** along with results. It clearly describes how the selected algorithm works and provides a **justification** for its selection. This enhances **conceptual clarity** and helps users understand the reasoning behind algorithm choice.

Overall, the results indicate that the system successfully automates the process of **algorithm selection** and **execution**. It reduces **manual effort**, improves **accuracy**, and provides meaningful insights into **algorithm performance**. The project achieves its objective of combining **problem-solving** with **conceptual learning**, making it a valuable tool for both academic and practical use.

CHAPTER 5
SUMMARY

This project, “Performance Evaluation of Scheduling, Memory, and Disk Algorithms in Operating Systems,” provides a comprehensive study of how different operating system algorithms manage system resources and influence overall performance. The work focuses on three major areas: CPU scheduling, memory management, and disk scheduling, each of which plays a critical role in ensuring efficient system operation. Through the implementation and simulation of various algorithms such as FCFS, SJF, and Round Robin for CPU scheduling; FIFO, LRU, and Optimal for memory management; and FCFS, SSTF, and SCAN for disk scheduling, the project evaluates their behavior under different conditions. Key performance metrics such as waiting time, turnaround time, response time, page faults, and seek time are used to measure and compare the efficiency of each algorithm. The analysis shows that different algorithms perform better in different scenarios. For example, SJF minimizes average waiting time but may lead to starvation of longer processes, while Round Robin ensures fairness among processes at the cost of increased context switching. In memory management, LRU generally provides better performance than FIFO by reducing page faults, whereas in disk scheduling, SSTF reduces seek time but may also cause starvation, and SCAN offers a more balanced approach. One of the key findings of this project is that there is no single “best” algorithm suitable for all situations. The effectiveness of an algorithm depends on factors such as system workload, process characteristics, and performance requirements. It also provides a strong foundation for further exploration in operating system design, optimization techniques, and real-world system implementation

CONCLUSION

The project “Performance Evaluation of Scheduling, Memory, and Disk Algorithms in Operating Systems” successfully demonstrates the importance of efficient resource management in an operating system. By implementing and analyzing various algorithms across CPU scheduling, memory management, and disk scheduling, this study provides a clear understanding of how different strategies impact overall system performance. In the area of CPU scheduling, algorithms such as FCFS, SJF, and Round Robin were evaluated using metrics like waiting time, turnaround time, and response time. The results indicate that while SJF offers optimal performance in terms of minimizing waiting time, it may lead to starvation of longer processes. Round Robin, on the other hand, ensures fairness and better responsiveness, especially in time-sharing systems, although it introduces additional overhead due to frequent context switching. FCFS, while simple and easy to implement, is less efficient in handling diverse workloads.In disk scheduling, algorithms like FCFS, SSTF, and SCAN were analyzed based on total seek time and head movement. FCFS is simple but inefficient due to larger seek times. SSTF reduces seek time by selecting the nearest request but may cause starvation for distant requests. SCAN, also known as the elevator algorithm, offers a balanced solution by servicing requests in a systematic direction, reducing overall movement and improving fairness. The overall conclusion drawn from this project is that no single algorithm is universally optimal for all conditions. This project not only strengthens the conceptual understanding of operating system algorithms but also provides practical insights into their real-world applications. It emphasizes the need for careful selection and adaptation of algorithms to achieve optimal system performance.

REFERENCES

[1] Operating System Concepts by Abraham Silberschatz, Peter B. Galvin, and Greg Gagne — a widely used textbook covering scheduling, memory, and disk algorithms in detail Modern Operating Systems — provides in-depth explanation of OS design, scheduling, and memory management concepts

[2] The Linux Programming Interface — explains practical OS concepts and system-level programming.

[3] GeeksforGeeks — useful for understanding algorithms like FCFS, LRU, and disk scheduling with examples.

[4] TutorialsPoint — provides simple explanations and implementations of OS concepts.

[5] Wikipedia — used for quick reference on scheduling algorithms and page replacement techniques